

# PySpark — Complete Reference

EY Probation Training — Week 1 (basics, DataFrames, transformations, aggregations, joins) and Week 4 (Databricks). Roadmap outcome bar is “understand and write basic PySpark code” — this goes further, because PySpark is also your professional differentiator.

## 1. Architecture in One Page

**Driver** runs your code, builds the DAG, and schedules work. **Executors** run tasks and hold cached data. The **Cluster Manager** (YARN, Kubernetes, Standalone, Databricks) allocates executors.

- **Job** — one action triggers one job.
- **Stage** — a job is split at shuffle boundaries.
- **Task** — one unit of work per partition within a stage.

**Catalyst** is the query optimiser: parsed plan → analysed plan → optimised logical plan → physical plan. **Tungsten** handles off-heap memory management and whole-stage code generation.

**Lazy evaluation:** transformations only build the DAG. Nothing executes until an action.

| Transformations (lazy)                                                                  | Actions (trigger execution)                                                                            |
|-----------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <code>select</code> , <code>filter</code> , <code>withColumn</code> , <code>drop</code> | <code>count</code> , <code>collect</code> , <code>show</code> , <code>first</code> , <code>take</code> |
| <code>groupBy</code> , <code>agg</code> , <code>join</code> , <code>orderBy</code>      | <code>write</code> , <code>foreach</code> , <code>toPandas</code>                                      |
| <code>distinct</code> , <code>repartition</code> , <code>union</code>                   | <code>cache</code> is lazy; <code>.count()</code> materialises it                                      |

| Narrow (no shuffle)                                                                                                                 | Wide (shuffle)                                                                                                                        |
|-------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| <code>select</code> , <code>filter</code> , <code>withColumn</code> , <code>map</code> , <code>union</code> , <code>coalesce</code> | <code>groupBy</code> , <code>join</code> , <code>distinct</code> , <code>orderBy</code> , <code>repartition</code> , window functions |

## 2. Session and Reading

```
from pyspark.sql import SparkSession, functions as F, Window
from pyspark.sql.types import *

spark = (SparkSession.builder
        .appName("app")
        .master("local[*]")
        .config("spark.sql.shuffle.partitions", "8")
        .getOrCreate())
spark.sparkContext.setLogLevel("WARN")
```

```
# Explicit schema – one pass, deterministic. Preferred in pipelines.
schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("order_date", DateType(), True),
    StructField("qty", IntegerType(), True),
    StructField("amount", DoubleType(), True),
])
df = spark.read.csv("f.csv", header=True, schema=schema)

# inferSchema costs an EXTRA full pass over the data
df = spark.read.csv("f.csv", header=True, inferSchema=True)

df = spark.read.option("escape", "'').csv(...) # when text fields contain quotes
df = spark.read.parquet("path/")
df = spark.read.json("path/")
df = spark.read.format("delta").load("path/")
```

**CSV quoting trap:** RFC-4180 escapes an inner quote by doubling it (""), but Spark’s CSV reader defaults to backslash. If a text column contains quotes, pass `escape='\"'` or fields silently misalign — and the corruption surfaces later as a baffling cast error.

## 3. Inspecting

```
df.show(20, truncate=False); df.printSchema(); df.columns; df.dtypes
df.count(); df.describe().show(); df.summary().show()
df.explain(); df.explain(True) # True = all four plans
df.rdd.getNumPartitions()
df.select([F.sum(F.col(c).isNull().cast("int")).alias(c) for c in df.columns]).show()
```

That last line gives null counts for every column in a single pass — much better than one action per column.

## 4. Columns and Expressions

```
df.select("a", "b")
df.select(F.col("a"), F.col("b").alias("bb"))
df.select(F.expr("qty * amount as revenue"))
df.selectExpr("qty * amount as revenue", "upper(region) as r")

df.withColumn("revenue", F.col("qty") * F.col("amount"))
df.withColumnRenamed("old", "new")
df.drop("a", "b")
df.filter(F.col("qty") > 5)
df.where("qty > 5 AND region = 'West'") # SQL string form
df.distinct(); df.dropDuplicates(["order_id"])
df.orderBy(F.desc("revenue"), F.asc("region"))
df.limit(10)
```

**Conditional logic** — the `when / otherwise` chain, evaluated top-down, first match wins:

```
F.when(F.col("amt") < 1000, "Low") \
  .when(F.col("amt") < 4000, "Medium") \
  .otherwise("High")
```

**Common functions:**

```
F.lit(0); F.coalesce(a, b, c); F.greatest(a,b); F.least(a,b)
F.round(x, 2); F.abs(x); F.floor(x); F.ceil(x); F.sqrt(x)
F.upper/lower/initcap/trim/ltrim/rtrim(col)
F.concat_ws("-", a, b); F.substring(c, 1, 5); F.length(c)
F.split(c, ","); F.regexp_replace(c, r"\s+", " "); F.regexp_extract(c, r"(\d+)", 1)
F.year/month/dayofmonth/quarter/hour/weekofyear(d)
F.date_format(d, "yyyy-MM"); F.datediff(a,b); F.add_months(d,1)
F.to_date(c, "yyyy-MM-dd"); F.to_timestamp(c, fmt); F.trunc(d, "month")
F.cast / col.cast("double")
```

**Null handling:**

```
df.na.drop(); df.na.drop(subset=["qty"]); df.na.drop(how="all")
df.na.fill(0); df.na.fill({"qty": 0, "region": "Unknown"})
df.na.replace(["W"], ["West"], "region")
F.col("x").isNull() / .isNotNull()
```

**Spark 4.x ANSI mode is ON by default.** A failed cast or timestamp parse now RAISES instead of returning null. Use `try_cast`, `try_to_timestamp` for version-independent null-on-failure behaviour. In Spark 3.x the same code silently produced nulls. Know which version you're on.

## 5. Aggregations

```
df.groupBy("region").count()
df.groupBy("region").agg(
  F.sum("revenue").alias("revenue"),
  F.avg("qty").alias("avg_qty"),
  F.countDistinct("sku").alias("skus"),
  F.approx_count_distinct("customer_id", 0.05).alias("approx_customers"),
  F.min("date").alias("first"), F.max("date").alias("last"),
  F.stddev("amount"), F.expr("percentile_approx(amount, 0.5)").alias("median"),
  F.collect_list("sku"), F.collect_set("sku"),
)
df.agg(F.sum("revenue")).first()[0] # scalar out

df.groupBy("region").pivot("quarter").agg(F.sum("revenue")) # Excel pivot equivalent
df.groupBy("region").pivot("quarter", ["Q1", "Q2"]).sum("revenue") # faster: values listed
df.rollup("region", "city").agg(F.sum("revenue")) # subtotals
df.cube("region", "channel").agg(F.sum("revenue")) # all combinations
```

`countDistinct` is exact but requires a shuffle; `approx_count_distinct` uses HyperLogLog and is far cheaper at scale when a small error is acceptable.

## 6. Joins

```
a.join(b, "key", "inner")           # string form: ONE merged key column
a.join(b, ["k1", "k2"], "left")    # multi-key
a.join(b, a.id == b.ref, "left")    # expression form: BOTH columns kept
a.join(F.broadcast(b), "key", "inner") # force broadcast
```

| Type         | Keeps                       | Notes                                       |
|--------------|-----------------------------|---------------------------------------------|
| inner        | Matching rows only          | Default                                     |
| left         | All left + matched right    | Right columns null when unmatched           |
| right        | All right + matched left    |                                             |
| outer / full | Everything                  |                                             |
| left_semi    | Left rows that HAVE a match | Left columns only; <b>cannot fan out</b>    |
| left_anti    | Left rows with NO match     | Left columns only; the integrity-check tool |
| cross        | Cartesian product           | Explicit only; guard carefully              |

Before every join, check two things:

```
dup = dim.groupBy("key").count().filter("count > 1").count() # is the key unique?
orphan = fact.join(dim.select("key").distinct(), "key", "left_anti").count()
assert dup == 0, "dimension key not unique -- join will fan out"
```

**Join fan-out** is the most damaging silent bug in Spark work. If the dimension key has duplicates, rows multiply and every downstream sum is overstated — no error, no warning. Assert row-count preservation on any join you expect to be 1:1.

**Column collision:** if both sides have a same-named non-key column, you get two columns and an `AMBIGUOUS_REFERENCE` error on any later reference. Drop or rename before joining.

**Broadcast join:** ships the small table to every executor, eliminating the shuffle. Automatic below `spark.sql.autoBroadcastJoinThreshold` (default 10 MB). The explicit hint matters when Spark's size estimate is wrong — after a filter, or with stale statistics. Look for `BroadcastHashJoin` vs `SortMergeJoin` in `.explain()`.

**Range join** (SCD type-2 / validity windows):

```
txn.join(prices, (txn.sku == prices.sku) &
              (txn.date >= prices.valid_from) &
              (txn.date <= prices.valid_to), "left")
```

Keep at least one equality in the condition. With no equi-key Spark falls back to `BroadcastNestedLoopJoin`, which is  $O(n \times m)$ .

## 7. Window Functions

```
w = Window.partitionBy("region").orderBy(F.desc("revenue"))

F.row_number().over(w)      # 1,2,3 – ties broken arbitrarily
F.rank().over(w)            # 1,2,2,4 – ties share, then SKIP
F.dense_rank().over(w)     # 1,2,2,3 – ties share, no skip
F.percent_rank().over(w)   # (rank-1)/(n-1)
F.ntile(4).over(w)         # quartile buckets
F.lag("x", 1).over(w); F.lead("x", 1).over(w)
F.first("x").over(w); F.last("x", ignorenulls=True).over(w)
```

**Frames — the most-tested distinction**

```
w.rowsBetween(-2, 0)           # 3 ROWS: previous 2 + current
w.rangeBetween(-1800, 0)      # 30 MINUTES of the ordering VALUE
w.rowsBetween(Window.unboundedPreceding, Window.currentRow) # cumulative
w.rowsBetween(Window.unboundedPreceding, Window.unboundedFollowing) # whole partition
w.rowsBetween(-3, 3)          # centred
```

- `rowsBetween` counts **rows**. `rangeBetween` counts **values of the ordering column** (needs a numeric/timestamp ordering key).
- With gaps in a time series these give different answers: 6 rows might span 50 minutes; 30 minutes might cover 3 rows.

**The last\_value trap:** the DEFAULT frame for an ordered window is `rangeBetween(unboundedPreceding, currentRow)`. So `F.last("x").over(w)` returns the CURRENT row, not the final one. You must specify `unboundedFollowing` explicitly. `first()` happens to work by accident, which is why this bug survives review.

**Common patterns:**

```

# top N per group
df.withColumn("rn", F.row_number().over(w)).filter("rn <= 3")

# deduplicate deterministically
df.withColumn("rn", F.row_number().over(
    Window.partitionBy("id").orderBy(F.desc("updated_at")))).filter("rn = 1")

# running total
F.sum("rev").over(Window.partitionBy("r").orderBy("d")
    .rowsBetween(Window.unboundedPreceding, Window.currentRow))

# share of group total
F.col("rev") / F.sum("rev").over(Window.partitionBy("region"))

# sessionization: flag -> lag -> cumulative sum
(df.withColumn("is_f", (F.col("status")=="FAULT").cast("int"))
    .withColumn("prev", F.lag("is_f",1,0).over(w2))
    .withColumn("start", ((F.col("is_f")==1)&(F.col("prev")==0)).cast("int"))
    .withColumn("run_id", F.sum("start").over(w2.rowsBetween(
        Window.unboundedPreceding, Window.currentRow))))

```

## 8. Performance

### Partitioning

```

df.rdd.getNumPartitions()
df.repartition(20)           # full shuffle, even sizes
df.repartition("store_id")   # hash by key – skewed keys land in one partition
df.coalesce(4)               # merges partitions, NO shuffle, can only REDUCE
df.withColumn("pid", F.spark_partition_id()).groupBy("pid").count()

```

`coalesce` can only decrease the count because creating new partitions requires redistributing rows — which is a shuffle. Use `coalesce` before a write to reduce output files; `repartition` when you need balance.

`spark.sql.shuffle.partitions` defaults to 200 regardless of data size. Rule of thumb: 2-3× total cores, or target ~128 MB per partition. AQE's `coalescePartitions` now adjusts this at runtime.

### Skew

Symptom: one task runs far longer than the rest — “99% done for 20 minutes”.

**Salting** — two-stage aggregation:

```

SALT = 16
(df.withColumn("salt", (F.rand()*SALT).cast("int"))
    .groupBy("key", "salt").agg(F.sum("v").alias("part"))
    .groupBy("key").agg(F.sum("part").alias("total")))

```

AQE handles skewed *joins* automatically by splitting the oversized partition:

```

spark.sql.adaptive.enabled           (true by default in 3.2+)
spark.sql.adaptive.skewJoin.enabled
spark.sql.adaptive.skewJoin.skewedPartitionFactor
spark.sql.adaptive.coalescePartitions.enabled

```

AQE does **not** fix a skewed `groupBy` on one hot key — salting is still your job.

### Caching

```

df.cache()           # = persist(MEMORY_AND_DISK); LAZY
df.persist(StorageLevel.MEMORY_ONLY)
df.count()           # materialise it
df.unpersist()

```

**When caching hurts:** used only once; larger than memory (MEMORY\_ONLY silently recomputes evicted partitions); starves execution memory; and it acts as an optimisation **barrier** — Catalyst can't push new filters through a cached DataFrame.

### Other

- Prefer Parquet over CSV: columnar, compressed, carries schema, per-rowgroup min/max stats enable block skipping.
- Avoid `collect()` — it pulls everything to the driver. `spark.driver.maxResultSize` guards against OOM. Use `take(n)`, `show()`, or write to storage.
- Filter early, select only needed columns — though Catalyst usually does this for you unless a UDF blocks it.

## 9. Writing

```
df.write.mode("overwrite").parquet("path/")
df.write.mode("append").partitionBy("year", "month").parquet("path/")
df.write.format("delta").mode("overwrite").save("path/")
df.write.bucketBy(8, "customer_id").saveAsTable("tbl")
df.coalesce(1).write.mode("overwrite").csv("path/", header=True)
```

| Mode          | Behaviour                                                   |
|---------------|-------------------------------------------------------------|
| overwrite     | Replaces the path                                           |
| append        | Adds files; watch for duplicates on retry                   |
| ignore        | Silent no-op if the path exists — dangerous, hides failures |
| errorifexists | Default; throws                                             |

`partitionBy` creates directories ( `channel=web/` ) and enables **partition pruning** — look for `PartitionFilters` in the plan. Partition on a LOW-cardinality column you actually filter on. Partitioning by date at day-grain on a small table creates the **small-files problem**: hundreds of tiny files where metadata and task-launch overhead dominate the actual read.

`bucketBy` hashes rows into a fixed number of files by key. Two tables bucketed identically on the same key join with **no shuffle**. Requires `saveAsTable` .

## 10. Spark SQL

```
df.createOrReplaceTempView("orders")           # this session only
df.createOrReplaceGlobalTempView("orders")      # all sessions; global_temp.orders
df.write.saveAsTable("orders")                  # persisted to the metastore

spark.sql("""
WITH q AS (
  SELECT channel, YEAR(order_date) yr, SUM(qty*amount) rev
  FROM orders GROUP BY channel, YEAR(order_date)
)
SELECT *, ROUND(100.0*(rev - LAG(rev) OVER (PARTITION BY channel ORDER BY yr))
               / LAG(rev) OVER (PARTITION BY channel ORDER BY yr), 1) AS yoy
FROM q ORDER BY channel, yr
""").show()
```

SQL and the DataFrame API compile to the **same** Catalyst plan — there is no performance difference. Choose for maintainability: SQL is portable and analyst-readable; the DataFrame API composes into functions and unit-tests cleanly.

## 11. UDFs — Last Resort

```
# 1. Plain Python UDF — SLOWEST
@F.udf(StringType())
def bucket(a): return "Low" if a < 1000 else "High"

# 2. pandas UDF — vectorised via Arrow
from pyspark.sql.functions import pandas_udf
@pandas_udf(StringType())
def bucket_pd(s: pd.Series) -> pd.Series:
    return pd.cut(s, [-1,1000,1e9], labels=["Low","High"]).astype(str)

# 3. Native expression — FASTEST
F.when(F.col("amt") < 1000, "Low").otherwise("High")
```

**Why the ordering:** a Python UDF serialises every row to a Python process and back, one at a time, and is a black box to Catalyst — no pushdown, no whole-stage codegen. A pandas UDF moves rows in Arrow batches with a columnar zero-copy layout, amortising the crossing. A native expression never leaves the JVM and gets folded into generated bytecode.

Native > pandas\_udf > python udf. Reach for a UDF only when no built-in expresses the logic.

## 12. Delta Lake (Databricks context)

```
df.write.format("delta").mode("overwrite").save(path)
spark.read.format("delta").option("versionAsOf", 3).load(path)    # time travel

from delta.tables import DeltaTable
dt = DeltaTable.forPath(spark, path)
dt.history().show()
dt.vacuum(168)                                                    # clean files older than 7 days
(dt.alias("t").merge(updates.alias("s"), "t.id = s.id")
 .whenMatchedUpdateAll().whenNotMatchedInsertAll()).execute()    # UPSERT
spark.sql("OPTIMIZE tbl ZORDER BY (customer_id)")
```

Delta adds ACID transactions, schema enforcement and evolution, time travel, and `MERGE` (upsert) on top of Parquet. `OPTIMIZE` compacts small files; `ZORDER` co-locates related data to improve skipping.

## 13. Practice Checklist

---

- ☐ Read with an explicit schema; explain why over `inferSchema`
- ☐ Null counts for all columns in one statement
- ☐ Clean dirty categoricals with `trim` / `initcap`
- ☐ Deduplicate with `row_number` and justify the ordering key
- ☐ Pre-join integrity checks: key uniqueness + `left_anti` orphan count
- ☐ All six join types with predicted row counts
- ☐ Force and verify a broadcast join in `.explain()`
- ☐ Multi-metric `agg()` plus a `pivot()`
- ☐ `row_number` / `rank` / `dense_rank` and explain the tie behaviour
- ☐ `rowsBetween` vs `rangeBetween` on data with gaps
- ☐ Detect skew, then fix it with salting; verify totals match
- ☐ Cache, time two actions, unpersist
- ☐ Write partitioned Parquet, confirm pruning in the plan
- ☐ Same query in SQL and DataFrame API; compare plans